

Fixed-K Speculative Decoding for Energy-Proportional LLM Serving: A Reproducible Empirical Validation

Zanno Jacklin

Independent Researcher, Creepybits

Abstract—We present an empirical validation of fixed draft-length ($K=5$) speculative decoding using a scout(1B)→target(8B) model pair on consumer GPU hardware. Across three prompt categories spanning low to high output determinism (creative poetry, technical explanation, and code generation), we measure throughput, energy per token, and output fidelity relative to a matched FP16 autoregressive baseline. Speculative decoding achieves up to +83.5% throughput improvement and −60.8% energy reduction per token on code generation, with smaller but consistent gains on less predictable prose (+8.4% / −32.7% on poetry). All results achieve 100% exact-token-match fidelity against the baseline (lossless, greedy decoding) and are independently reproducible from the accompanying open-source repository. We report results transparently, including the deliberate methodological choice to omit KV-caching from both arms of the comparison, which keeps the relative comparison fair but means absolute throughput figures are not representative of a production-optimized server, and including two documented measurement-methodology corrections (a warmup-timing bug, and subsequently a warmup convergence-criterion bug, both affecting earlier versions of this result) alongside the closed-loop warmup convergence and residual-drift diagnostics that caught them and now guard against a recurrence. In a supplementary benchmark (Section III-E), we additionally compare this same mechanism against vLLM’s own native `draft_model` speculative decoding under matched conditions, finding a +22% to +100% throughput improvement over vLLM’s own FP16 baseline when run natively inside vLLM’s serving engine.

Index Terms—speculative decoding, energy-efficient inference, large language models, GPU power measurement

I. INTRODUCTION

Autoregressive decoding in large language models (LLMs) is inherently sequential: each output token requires a full forward pass through the target model before the next token can be produced. This serial dependency underutilizes modern GPU compute, which is optimized for parallel throughput. Speculative decoding [1], [2] addresses this by using a smaller, faster *scout* model to propose a short window of candidate tokens, which the larger *target* model then verifies in a single parallel forward pass, accepting the longest correct prefix.

This paper reports a narrow, fully-validated empirical claim: a fixed draft window of $K=5$ tokens, paired with an exact (lossless) accept/reject verification scheme, produces measurable throughput and energy improvements on real consumer GPU hardware, with perfect output fidelity relative to standard greedy decoding. We deliberately scope this paper to only the mechanism we have fully validated. A related, more

ambitious mechanism under investigation in the parent project — entropy-gated *dynamic* adjustment of the draft window, intended to switch between aggressive and conservative speculation based on local output entropy — is explicitly **not** included here, as it has not yet been shown to outperform this simpler fixed- K approach in single-request testing.

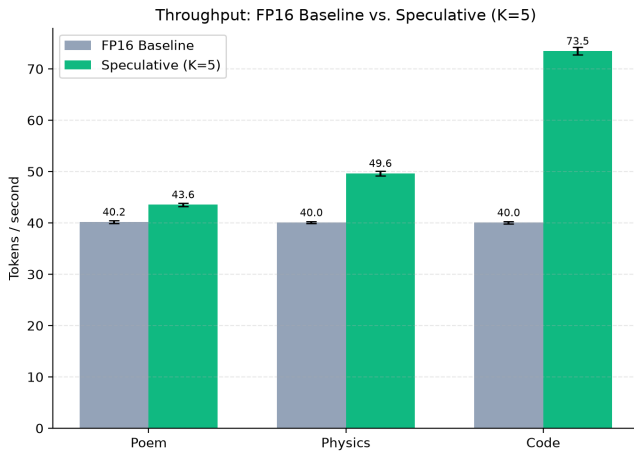
A. Relationship to prior work

Entropy-gated adaptive speculative decoding is an active research area. AdaEDL [3] reports 10–57% improvement over static draft length using entropy-based adaptive drafting. SGLang ships a built-in adaptive speculative-length mechanism using an EMA of accepted draft length — a different mechanism in kind from the fixed- K approach reported here, closer in spirit to the entropy-gated dynamic mechanism excluded from this paper (see Introduction) than to fixed- $K5$ itself; it is not benchmarked in this paper. vLLM, by contrast, ships a `draft_model` speculative-decoding method that is mechanistically the same approach as this paper’s own (small draft model, fixed draft window, target-model verification) — Section III-E reports a direct head-to-head against it. Our own testing of an entropy-gated variant of *our* mechanism, using the same hardware and methodology as this paper, has not yet demonstrated an improvement over the simpler fixed- K approach reported here; we regard that as an open question requiring further work (e.g. testing under multi-request/batched serving, which published entropy-gating results suggest is where the technique’s value proposition may actually lie), not a settled negative result. This paper makes no claims about that entropy-gated mechanism in either direction.

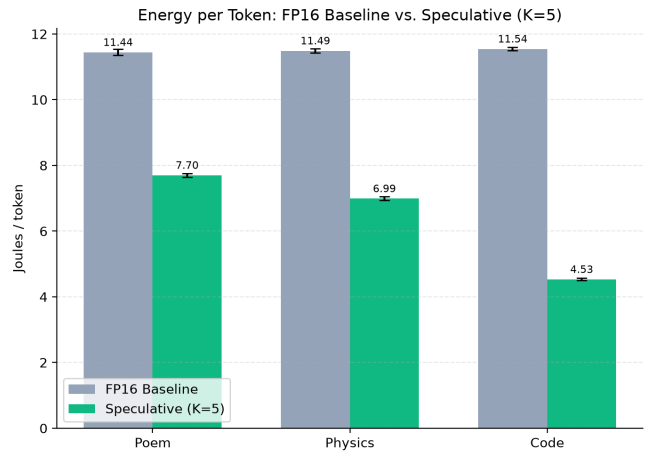
II. METHODOLOGY

A. Models and hardware

Scout model: Llama-3.2-1B-Instruct. Target model: Llama-3.1-8B-Instruct. Both loaded in bfloat16. All measurements taken on a single NVIDIA RTX 5090 (Blackwell architecture) GPU, single-request (batch size 1), greedy (argmax) decoding throughout. The results reported here were collected under WSL2, without NVML GPU clock locking (no elevated privileges available in that environment); this is noted for reproducibility, since clock-locked runs would be expected to show lower run-to-run variance.



(a) Throughput



(b) Energy per token

Fig. 1: FP16 baseline vs. fixed- $K=5$ speculative decoding, per prompt ($N=10$ trials/prompt, error bars show ± 1 std. dev.). Gains are smallest on the least predictable content (Poem) and largest on the most predictable (Code).

B. Draft-verify procedure

At each decoding cycle, the scout model autoregressively proposes $K=5$ candidate tokens. The target model then performs a single forward pass over the full draft sequence, and its argmax predictions at each position are compared against the corresponding drafted tokens. The longest matching prefix is accepted; at the first mismatch (or after all K tokens match), the target’s own prediction is substituted, exactly reproducing what standard greedy decoding would have produced at that position. This verification scheme is lossless by construction: the output token sequence is provably identical, position-for-position, to what running the target model alone would produce, since every emitted token is either a drafted token confirmed by the target’s own argmax, or the target’s argmax directly.

C. Power and energy measurement

GPU power is sampled via NVML at 100 Hz on a background thread throughout each timed generation window. Energy per token is read from the GPU’s onboard hardware energy counter (`nvmlDeviceGetTotalEnergyConsumption`) when available, as it was for every trial reported in this paper, divided by N_{tok} , the number of tokens produced. This is a more direct measurement than integrating sampled instantaneous power readings, which is retained in the accompanying code only as a fallback and cross-check for hardware that lacks the counter.

D. Warmup

Warmup is applied in two layers. First, a one-time *closed-loop thermal warmup* runs real, discarded decoding cycles – alternating across all prompts and both the baseline and speculative conditions – until, for each individual prompt/condition combination separately, both its power

reading and its die-temperature reading stop drifting relative to their own recent history. Checking per-combination rather than pooled matters for two independent reasons found in sequence. One: baseline and speculative decoding draw genuinely different power by design; a convergence check that simply requires consecutive readings to agree with each other, without accounting for which workload produced them, can fail to terminate when the workload alternates between conditions with a real, persistent power gap. The other, found only after fixing the first issue: die temperature is not actually workload-independent on this hardware – baseline and speculative decoding settle at temperatures roughly 3–4°C apart, so pooling temperature across both (on the assumption that it is a slow, workload-independent signal) produces a permanent oscillation in the rolling window that no absolute tolerance can ever satisfy, regardless of run length. A separate, independent issue affected the power check alone: an absolute tolerance of 1.5 W was tighter than the hardware’s own sample-to-sample noise floor (measured standard deviation 2.9 W at baseline load, 6.8 W at speculative load), making it unsatisfiable on its own terms. All three are corrected in the procedure used for this paper: warmup burst length matches the real trial length (an earlier version used a shorter burst, which was found empirically to converge at a lower power/thermal level than the real, longer trial then reached); power and temperature are both tracked per prompt/condition combination rather than pooled; and convergence for both is judged by a half-split drift statistic (the difference between the mean of the newer half and the older half of a rolling window) against a tolerance relative to each combination’s own mean (1.5% for power, 1°C absolute for temperature, empirically already well above its own noise floor), rather than raw min-max spread, which grows with window length under pure noise and does not isolate drift from noise the way a half-split comparison

does. Second, independently of the closed-loop warmup above, each individual timed trial is still preceded by 5 short untimed forward-pass warmup steps immediately before its own measurement window, avoiding cold-SM effects specific to that trial. This two-layer procedure is shared between both arms of the comparison (`benchmark_ablation.py` and `speculative_scout.py`) via a common `bench_common.py` module, and reused by the supplementary vLLM benchmark in Section III-E.

E. Deliberate exclusion of KV-caching

Both the baseline and speculative decoding paths recompute attention over the full sequence at every step, with no KV-cache reuse. This was a deliberate methodological choice: an earlier iteration of this work applied caching only to the baseline arm, which structurally penalized the speculative path whenever it fell back to single-step generation (a full uncached recompute for a single token is the worst-case cost ratio in the system). Removing caching from both arms restores a fair relative comparison at the cost of both arms being substantially slower in absolute terms than a production server with caching would be. We therefore report *relative* throughput and energy figures as the validated result, and explicitly flag that absolute tok/s numbers in this paper are not production-representative. Section III-E reports a separate, KV-cached comparison as a distinct, non-substitutable result.

F. Fidelity measurement

Fidelity is measured as the exact token-for-token match rate between the baseline’s greedy output and the speculative path’s output, over the full generated sequence, for each prompt.

G. Prompts

Three prompts were selected to span a range of output determinism:

- **Poem** (low determinism): an open-ended creative writing task (an original Chant Royal poem on a mathematical theme).
- **Physics** (medium determinism): a technical explanation task (semiconductor memory bandwidth and the memory wall).
- **Code** (high determinism): a code-generation task (a Python binary search tree implementation with type annotations), where correct completions are far more constrained token-to-token.

III. RESULTS

A. Fidelity

Aggregate exact-token-match fidelity across all three prompts was 100.0%, confirming the accept/reject implementation is lossless in practice as well as by construction.

B. Throughput, energy, and power

Table I reports mean $\pm$ standard deviation across $N=10$ trials per prompt per condition.

Mean GPU power during speculative decoding (346–364 W) is lower than during FP16 baseline decoding (463–466 W), despite two models being resident in VRAM simultaneously. This is consistent with fewer full 8B-parameter forward passes being required per unit of output as accepted draft batch length grows – each accepted cycle of up to $K+1=6$ tokens requires only one target forward pass, versus six for the baseline. Accept rates (42.5% / 51.7% / 85.4%) and per-prompt throughput/energy figures both agree closely with an independent single-run measurement from `speculative_scout.py` (a separate script sharing only `bench_common.py`, not the trial-averaging logic), which we take as further evidence against measurement artifacts specific to either script.

C. Measurement validity: residual drift diagnostics

After the corrections to the warmup procedure described in Section II-D, each run’s per-trial telemetry is checked for residual drift: a least-squares fit of power, energy/token, and throughput against chronological trial index within the run. The run behind Table I is the first on this project with a closed-loop warmup that fully converged within its time cap (241.3 s, well under the 420 s cap; every prior run had hit the cap without converging) and the cleanest drift diagnostics observed to date: the fitted trend was negligible for every metric, pooled and per-condition (R^2 between 4×10^{-5} and 0.037 across all cases), and the accompanying code’s pass/fail threshold ($\pm 0.5\%$ of the mean) passed cleanly on every check (pooled 0.30%; baseline-only 0.38%; speculative-only 0.19%) – unlike earlier runs, where the threshold fired despite a near-zero R^2 and had to be interpreted rather than simply read off. We treat the results in Table I as the most reliable of this project to date on that basis.

D. Speedup and energy reduction scale with acceptance rate

Figure 2 shows speedup tracking draft acceptance rate across the three prompts, consistent with the mechanism’s expected behavior: more predictable continuations (code) allow the scout to correctly anticipate more tokens per cycle, directly translating to fewer expensive target forward passes.

E. Comparison to vLLM’s Native Speculative Decoding

vLLM ships its own native speculative decoding, including a `draft_model` method with a fixed `num_speculative_tokens` setting – mechanistically the same approach as this paper’s own (small draft model, fixed draft window, target model verifies). As a supplementary benchmark, we compare this paper’s fixed- $K=5$ mechanism directly against it: the same scout (Llama-3.2-1B-Instruct) and target (Llama-3.1-8B-Instruct) models, the same three reference prompts, vLLM’s own engine and KV-cache for both arms, $N=10$ trials per prompt, greedy decoding. Table II reports the result.

TABLE I: Baseline vs. fixed- $K=5$ speculative decoding, per prompt. N=10 trials/prompt. Collected 2026-09-05, the first run with a fully converged closed-loop warmup (see Section III-C).

Prompt	Baseline tok/s	Spec. tok/s	Speedup	Baseline J/tok	Spec. J/tok	Energy Δ	Accept %
Poem	40.16 $\pm$ 0.28	43.55 $\pm$ 0.31	+8.4%	11.44 $\pm$ 0.09	7.70 $\pm$ 0.06	-32.7%	42.5%
Physics	40.05 $\pm$ 0.18	49.64 $\pm$ 0.43	+24.0%	11.49 $\pm$ 0.06	6.99 $\pm$ 0.06	-39.2%	51.7%
Code	40.04 $\pm$ 0.23	73.46 $\pm$ 0.74	+83.5%	11.54 $\pm$ 0.05	4.53 $\pm$ 0.04	-60.8%	85.4%

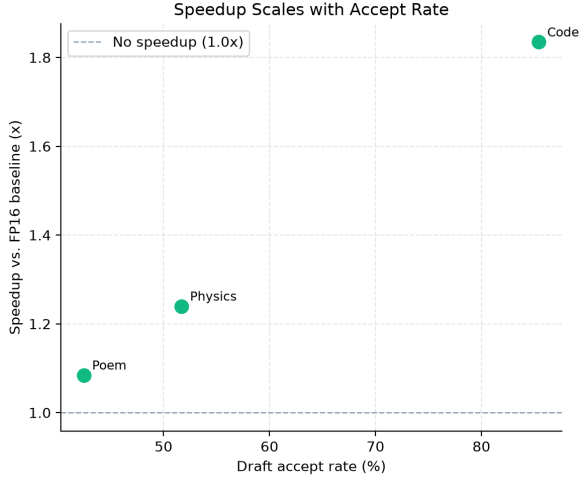


Fig. 2: Speedup vs. FP16 baseline as a function of draft accept rate, across the three prompts. The relationship is monotonic and consistent with the mechanism’s expected behavior.

TABLE II: Fixed- $K=5$ mechanism vs. vLLM’s own native speculative decoding, run natively inside vLLM (both arms KV-cached). N=10 trials/prompt.

Prompt	vLLM base. tok/s	vLLM+K5 tok/s	Speedup
Poem	100.6	123.1	+22.4%
Physics	100.8	151.8	+50.6%
Code	100.8	201.4	+99.9%

The fixed- $K=5$ mechanism measurably outperforms vLLM’s own FP16 baseline when run natively inside vLLM’s serving engine, across all three reference prompts. Three caveats apply. First, this result is **not directly comparable** to Table I: vLLM uses PagedAttention (KV-caching) on both arms here, whereas Table I deliberately omits caching on both arms for the different reason described in Section II-D (isolating the speculative mechanism from caching effects) – these are two distinct, both methodologically valid comparisons answering different questions, and their baseline numbers should not be read as points on the same axis. Second, this benchmark covers vLLM only, not SGLang; SGLang’s own built-in speculative decoding is adaptive (EAGLE-based, tiered draft length), a different mechanism closer in kind to the entropy-gated dynamic approach this paper explicitly excludes than to fixed- $K=5$, and has not been benchmarked here. Third, this is a single full run, not yet independently cross-validated by a second script the way

Table I is corroborated by `speculative_scout.py`; the same closed-loop, per-prompt warmup convergence procedure described in Section II-D was reused for this benchmark (via the shared `bench_common.py` module), and a first-round rotation-order artifact found during development of this benchmark (whichever prompt ran first in a trial round measured slower, independent of which specific prompt it was) was diagnosed and fixed using that same per-label convergence check before the result above was collected.

IV. DISCUSSION AND LIMITATIONS

A. Absolute throughput is not production-representative

As noted in Methodology, neither arm of the main comparison (Table I) uses KV-caching. A production serving stack with proper cache reuse would show substantially higher absolute throughput in both arms. Section III-E’s vLLM comparison does use a production-representative KV-cached serving path, but is a distinct benchmark against a different baseline (vLLM’s own), not a KV-cached extension of Table I; a KV-cached variant of this paper’s own manual ablation harness, directly comparable to Table I, remains future work and is the primary remaining gap before these results can inform a deployment decision in full.

B. Single-request, single-GPU scope

All results in this paper are single-request (batch size 1) measurements on a single consumer GPU. Behavior under batched, multi-request serving – where the entropy-gated dynamic variant may plausibly outperform this fixed- K approach, per precedent in AdaEDL and SGLang – is untested and out of scope for this paper.

C. Reproducibility

All code, raw telemetry (JSON/CSV), and the scripts used to generate every figure and table in this paper are available at <https://github.com/Creepybits/sdsie-fixed-k5-speculative-decoding> under an Apache-2.0 license. Reported numbers in Table I are directly traceable to `telemetry/ablation_results.json`, generated by `benchmark_ablation.py` in that repository; Table II is directly traceable to `telemetry/vllm_comparison_results.json`, generated by `benchmark_vllm_comparison.py`. The closed-loop warmup and drift-diagnostic logic described in Sections II-D and III-C is shared between all three scripts via `bench_common.py`.

V. CONCLUSION

Fixed- $K=5$ speculative decoding, with an exact lossless verification scheme, produces measurable and reproducible throughput and energy gains on real consumer GPU hardware, scaling with output predictability, with zero loss of output fidelity. A supplementary benchmark further shows this same mechanism outperforming vLLM’s own native speculative decoding by +22% to +100% when run inside vLLM’s serving engine (Section III-E), though that comparison remains a single run pending independent cross-validation and does not cover SGLang. We report this as a validated, narrow result, distinct from and not dependent on the more experimental entropy-gated dynamic mechanism under investigation elsewhere in the parent project.

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” arXiv:2302.01318, 2023.
- [3] Qualcomm AI Research, “AdaEDL: Adaptive Entropy-based Dynamic Layer for Efficient Speculative Decoding,” arXiv:2410.18351, 2024.